
Cấu trúc dữ liệu và giải thuật

IT003.P21.CTTN

Assignment 4.01

Họ và tên	Lê Hữu Hoàng + Chu Quang Cường
MSSV	24520539 + 24520236
Lớp	ATTN2024

Link github:

Hoàng: <https://github.com/hhthuis/DLL---BST/tree/main>

Cường: https://github.com/R1MURUN0PR0/DSA_UIT/tree/main/Assignment04

a/ Viết hàm chuyển cây BST sang Doubly Linked List tăng.

Xây dựng hàm:

Hàm BSTToSortedDLL (Chuyển BST thành DLL tăng):

- Tham số: root (con trỏ đến nút gốc của cây BST)
 - Biến:
 - curr: con trỏ đến nút hiện tại, bắt đầu từ root.
 - prev: con trỏ đến nút trước đó, ban đầu là nullptr.
 - final_head: con trỏ đến nút đầu của DLL, ban đầu là nullptr.
 - Vòng lặp chính: Trong khi curr không phải nullptr:
 - Trường hợp 1: Nếu curr không có con trái:
 - Nếu final_head là rỗng, ta gán final_head = curr (đặt nút đầu tiên).
 - Gán prev = curr và chuyển curr sang con phải: curr = curr→right.
 - Trường hợp 2: Nếu curr có con trái:
 - Tìm nút phải nhất trong cây con trái của curr:
 - ❖ Bắt đầu từ pre = curr→left.
 - ❖ Trong khi pre→right tồn tại và không trỏ đến curr, di chuyển pre sang phải: pre = pre→right.
 - Trường hợp 2.1: Nếu pre→right là nullptr:
 - ❖ Tạo liên kết tạm thời: pre→right = curr.
 - ❖ Di chuyển curr sang con trái: curr = curr→left.
 - Trường hợp 2.2: Nếu pre→right đã trỏ đến curr:
 - ❖ Di chuyển curr sang con phải: curr = curr→right.
 - ❖ Liên kết prev và curr trong DLL: prev→right = curr, curr→left = prev.
 - ❖ Cập nhật prev = curr.
 - ❖ Tiếp tục di chuyển curr sang con phải: curr = curr→right.
 - Kết thúc: Trả về final_head, là nút đầu tiên của DLL.
-

Ý tưởng chính: Sử dụng [Morris Traversal](#) để duyệt cây theo thứ tự **inorder**.

Script:

```
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

Node* BSTToSortedDLL(Node* root) {
    Node* curr = root;
    Node* prev = nullptr;
    Node* final_head = nullptr;

    while (curr) {
        if (!curr->left) {
            if (!final_head) {
                final_head = curr;
            }
            prev = curr;
            curr = curr->right;
        } else {
            Node* pre = curr->left;

            while (pre->right && pre->right != curr) {
                pre = pre->right;
            }

            if (!pre->right) {
                pre->right = curr;
                curr = curr->left;
            } else {
                curr = curr->right;
                prev->right = curr;
                curr->left = prev;
                prev = curr;
                curr = curr->right;
            }
        }
    }

    return final_head;
}
```

b/ Viết hàm chuyển Doubly Linked List tăng sang cây BST.

Xây dựng hàm: Ở đây ta sẽ xây dựng hai hàm hỗ trợ xây dựng cây và in ra.

Hàm sortedDLLToBST (Chuyển DLL tăng thành BST):

- Tham số:
 - head: tham chiếu đến con trỏ của nút đầu tiên trong DLL.
 - n: số lượng nút trong DLL.
- Logic:
 - Nếu $n \leq 0$, trả về nullptr do DLL rỗng hoặc không có nút.
 - Độ quy để xây dựng cây con bên trái với $\frac{n}{2}$ nút đầu tiên của danh sách để tạo cây con trái.
 - Tạo nút gốc (root) với giá trị ở nút hiện tại trong danh sách.
 - Gán cây con trái cho root.
 - Di chuyển head sang nút tiếp theo trong danh sách.
 - Độ quy để xây dựng cây con bên phải với $n - \frac{n}{2} - 1$ nút còn lại để tạo cây con phải.
 - Trả về nút gốc (root).

Hàm inorder (Duyệt cây BST theo thứ tự inorder):

- Tham số:
 - root: con trỏ đến nút gốc của cây BST.
- Logic:
 - Nếu root là nullptr, thoát hàm.
 - Độ quy để duyệt cây con bên trái.
 - In giá trị của nút hiện tại.
 - Độ quy để duyệt cây con bên phải.

Ý tưởng chính: Hàm sortedDLLToBST chia DLL thành hai phần đệ quy và tạo ra cây BST có thứ tự khớp với danh sách ban đầu. Hàm inorder dùng để in cây theo thứ tự tăng dần.

Script:

```
struct Node {
    int data;
    Node* left;
    Node* right;
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

Node* sortedDLLToBST(Node* &head, int n) {
    if (n <= 0) return nullptr;

    Node* left = sortedDLLToBST(head, n / 2);
    Node* root = new Node(head->data);
    root->left = left;
    head = head->right;
    root->right = sortedDLLToBST(head, n - n / 2 - 1);

    return root;
}

void inorder(Node* root) {
    if (root == nullptr) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```
